EEE 391 - MATLAB Mini Project 2

Irmak İmdat - - Section 1

Introduction

In this Mini Project assignment, reconstruction of a signal with different sequences, different pulses, and different sampling rates is observed. There is two versions of the same code. The published one does not take user input since publish function does not allow it, the second version which is attached at the end of the paper, is a user interactive version of the same functionality that allows the user to choose Ts, pulse type, and sequences (since the assignment states "The program should be able to take any sequence y[n] of finite duration and the sampling period Ts as input.").

The following code is for initializing Ts and sequence values.

```
clear
clc
close all

Ts = 1;

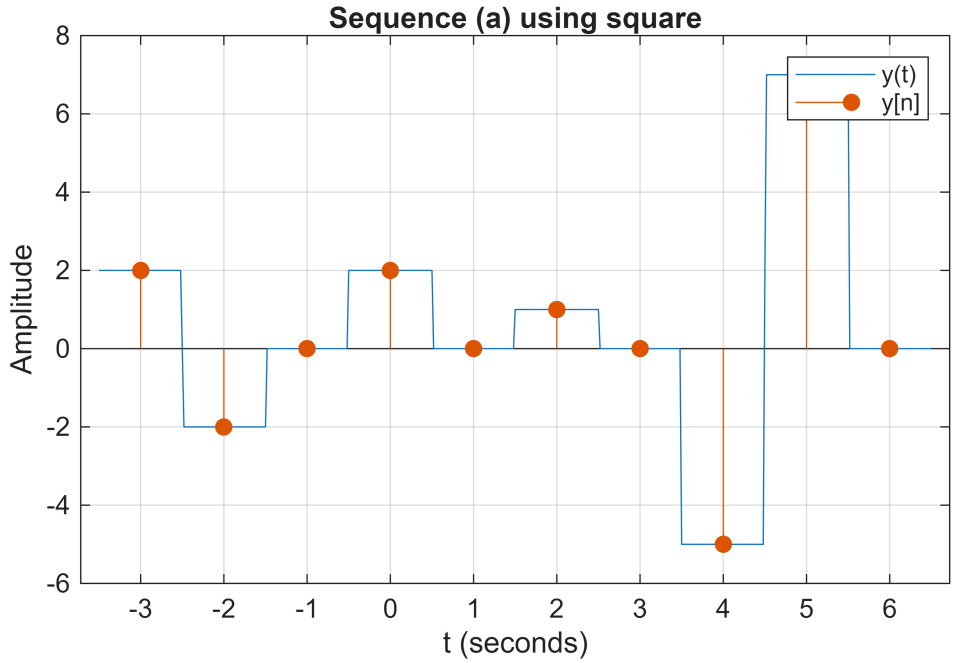
n_a = -3:6;
y_a = [2, -2, 0, 2, 0, 1, 0, -5, 7, 0];

n_b = -12:12;
y_b = 10 * exp(-0.25 * abs(n_b));
```

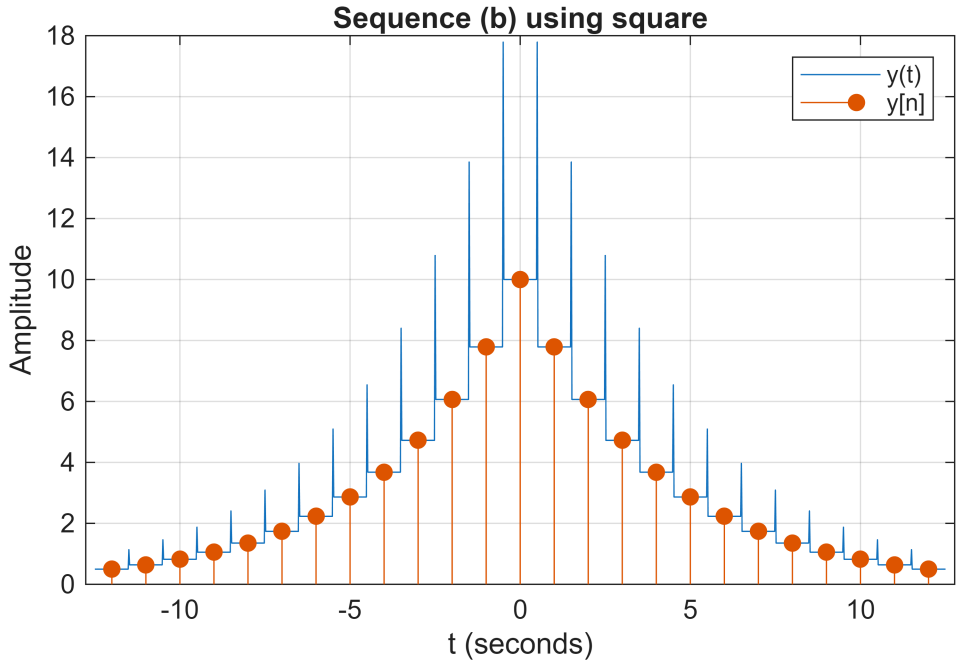
Square Pulse

The following code calls *reconstruct_and_plot* function to plot sequences a and b with square pulse. You can see that the reconstructed signal takes the form of a "staircase." This is because the square pulse holds the amplitude of each sample constant for the duration of the sampling period, this results in a zero-order hold reconstruction with discontinuities at the sample boundaries.

```
reconstruct_and_plot(n_a, y_a, Ts, 'square', 'Sequence (a)');
```



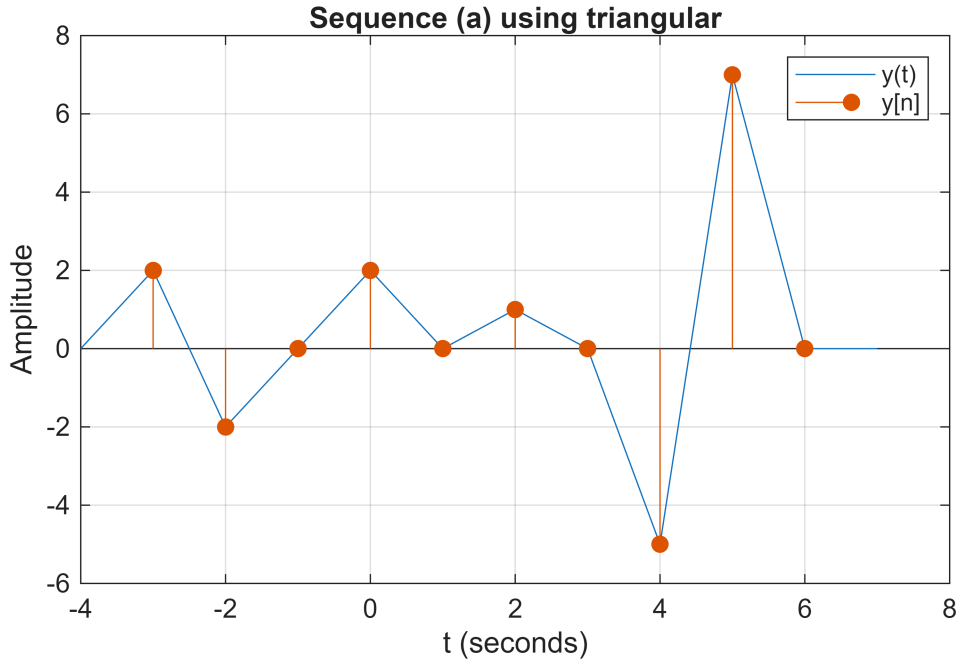

```
reconstruct_and_plot(n_b, y_b, Ts, 'square', 'Sequence (b)');
```



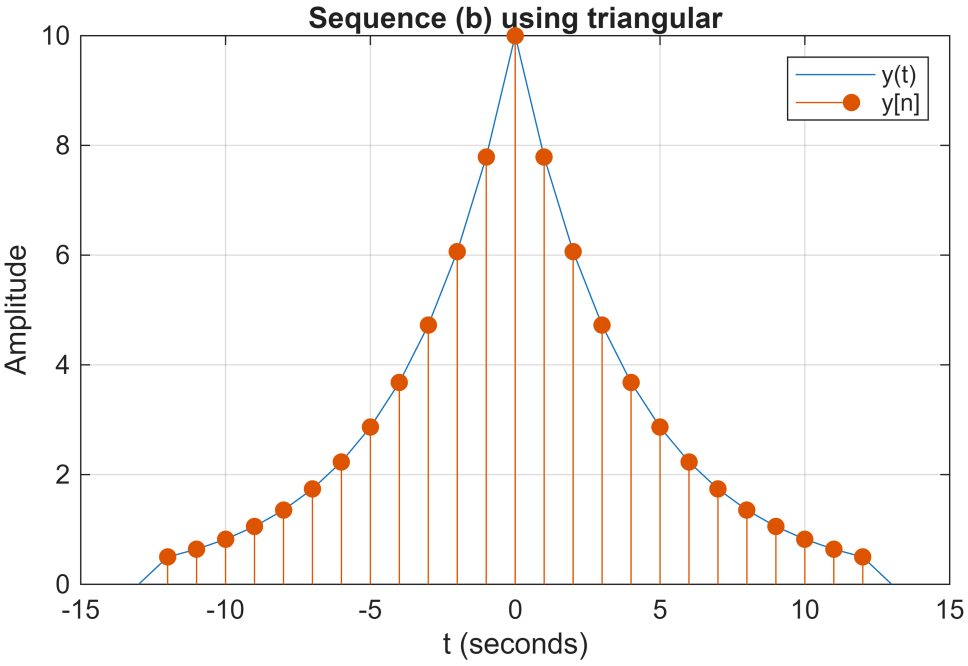
Triangular Pulse

The following code calls *reconstruct_and_plot* function to plot sequences a and b with triangular pulse. This pulse performs linear interpolation and connects the sample points with straight lines. As you can see, choosing this pulse type avoids the discontinuities seen in the square pulse, however, the reconstructed signal still has sharp corners which causes discontinuous derivatives.

```
reconstruct_and_plot(n_a, y_a, Ts, 'triangular', 'Sequence (a)');
```




```
reconstruct_and_plot(n_b, y_b, Ts, 'triangular', 'Sequence (b)');
```




```

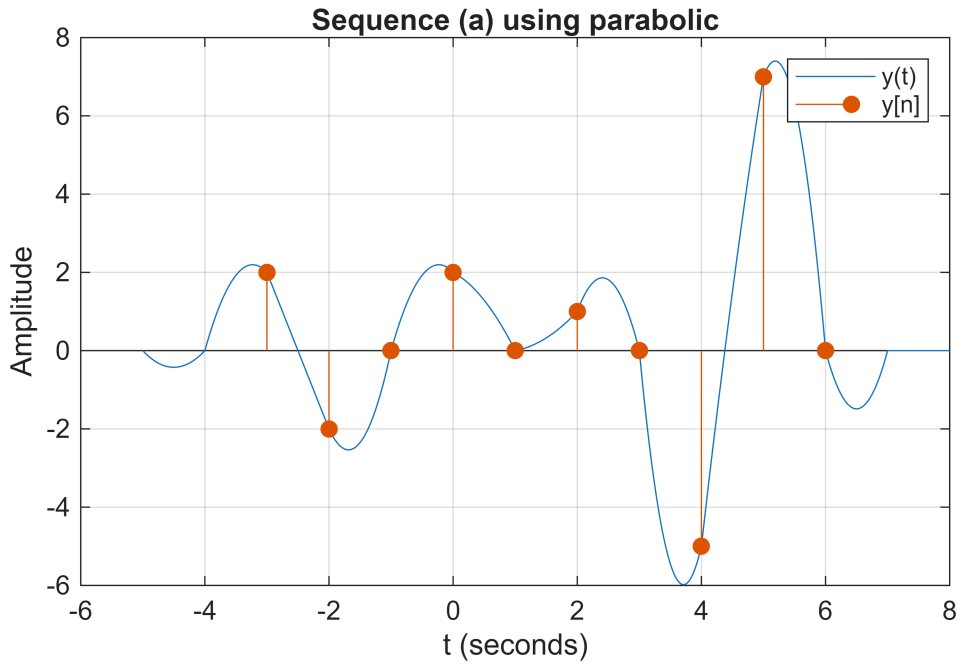
```

Parabolic Pulse

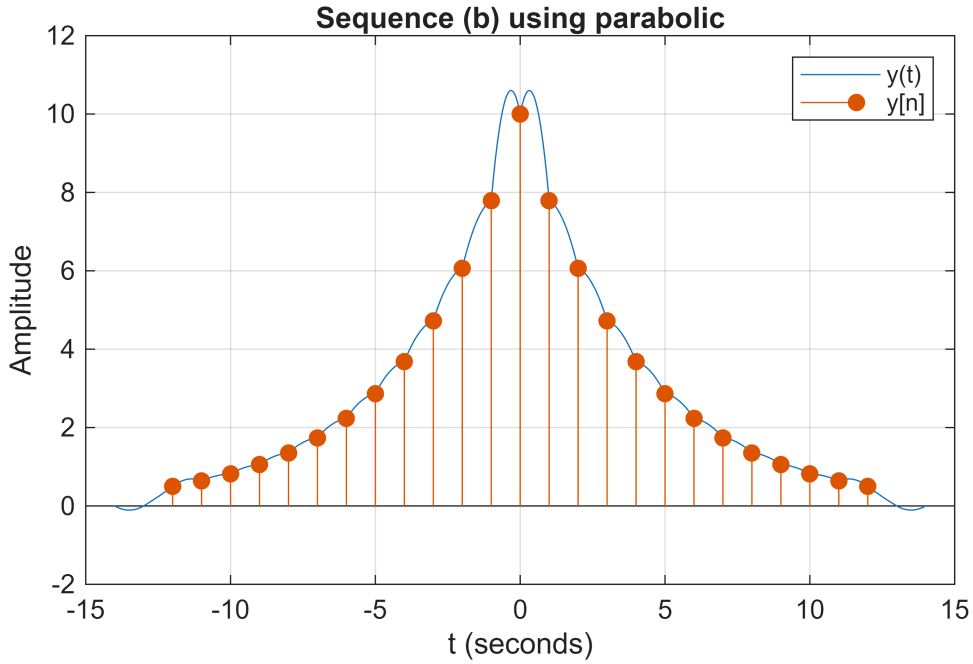
The following code calls the *reconstruct_and_plot* function using the parabolic pulse. Before, the triangular pulse created sharp peaks with undefined first derivatives, but this pulse creates a smooth curve. The side

segments of this pulse are designed to dip below zero so that it resembles the negative side-lobes of the ideal Sinc function. This provides a better approximation of band-limited reconstruction than the simple linear interpolation of the triangular pulse.

```
reconstruct_and_plot(n_a, y_a, Ts, 'parabolic', 'Sequence (a)');
```



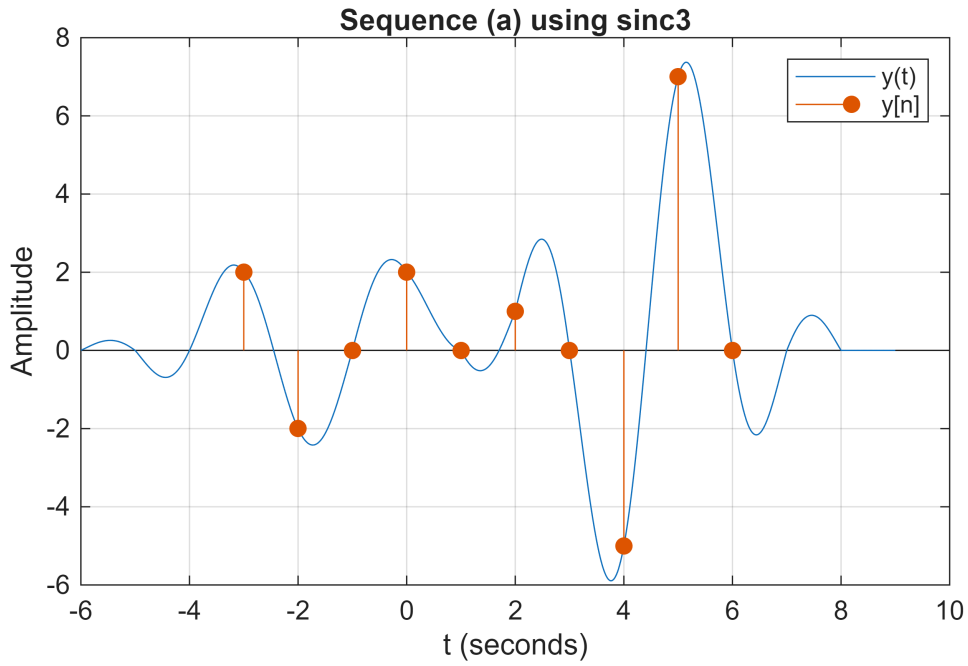

```
reconstruct_and_plot(n_b, y_b, Ts, 'parabolic', 'Sequence (b)');
```



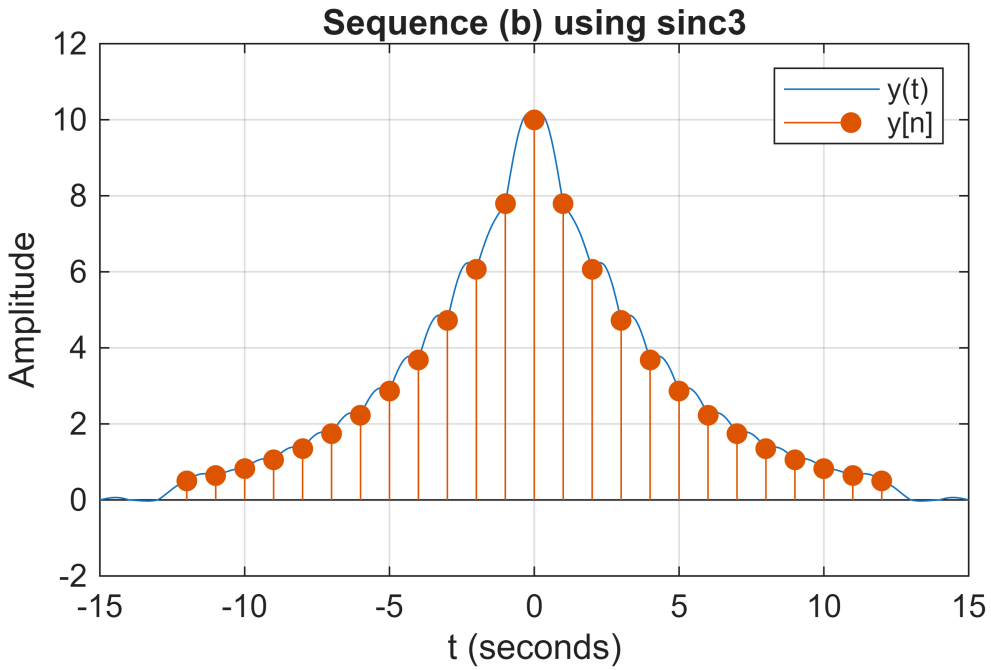
Sinc3 and Sinc5 Pulses

The following code calls *reconstruct_and_plot* function to plot sequences a and b with Sinc pulse. Since the ideal Sinc function extends to infinity, it must be truncated.

```
reconstruct_and_plot(n_a, y_a, Ts, 'sinc3', 'Sequence (a)');
```

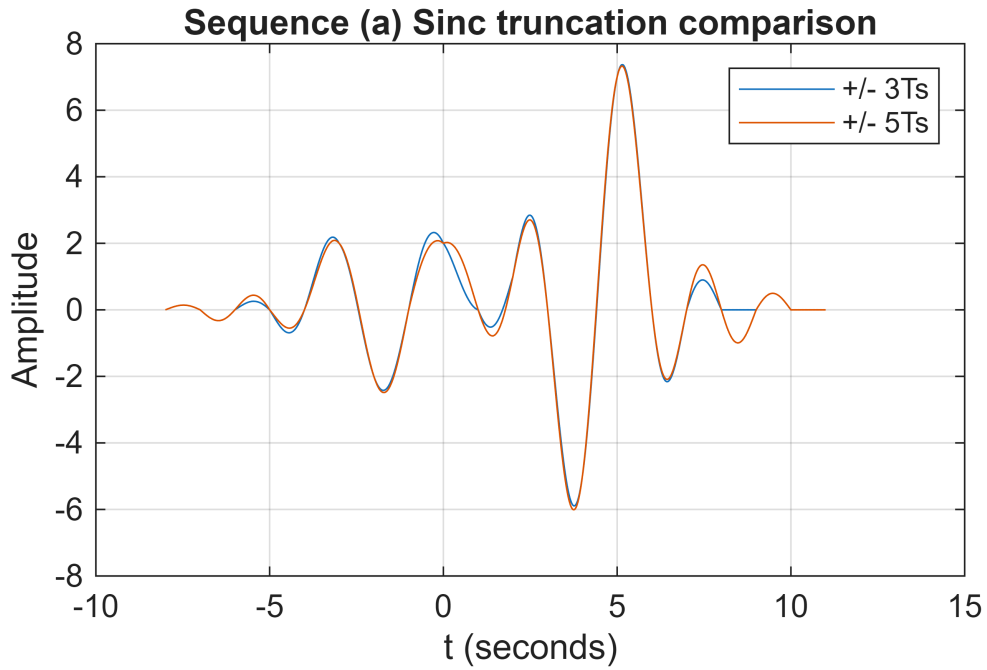



```
reconstruct_and_plot(n_b, y_b, Ts, 'sinc3', 'Sequence (b)');
```

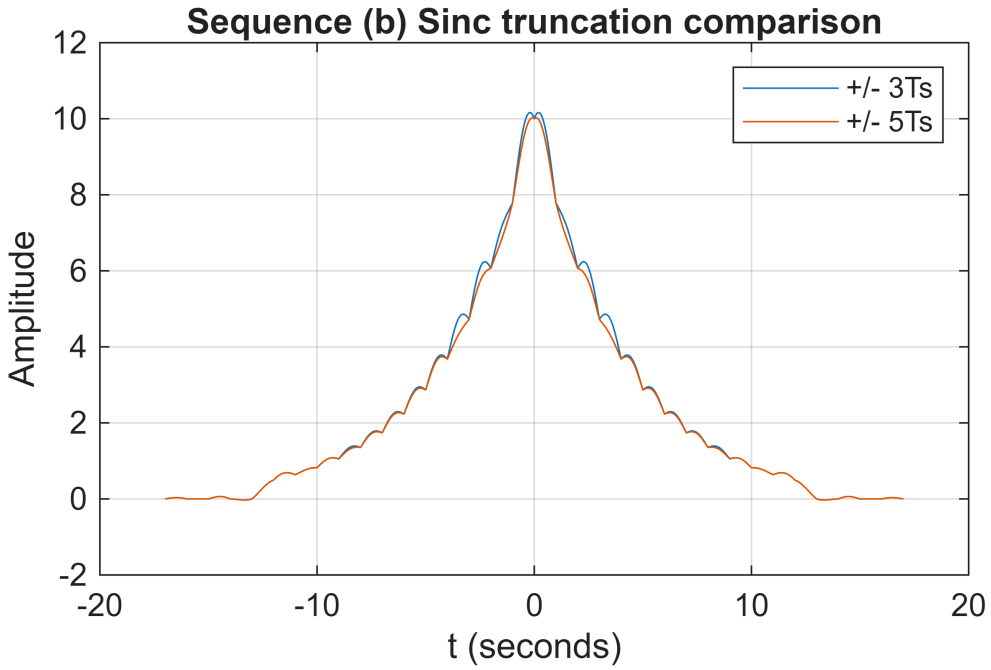


To see the effect of truncaton width, different reconstructions with 3Ts and 5Ts are plotted on the same figure. As you can see, the plot with the wider pulse (5Ts) results in a better reconstruction. This is because of the additional side lobes of 5Ts that help capturing the ideal Sinc function better.

```
compare_sinc_truncation(n_a, y_a, Ts, 'Sequence (a)');
```




```
compare_sinc_truncation(n_b, y_b, Ts, 'Sequence (b)');
```



Reconstruction and Plotting

This function first calls *reconstruct_signal* function, then uses it's information for plotting the reconstructed signal.

```
function reconstruct_and_plot(n, y, Ts, pulse_type, seq_label)
    if strcmp(pulse_type, 'sinc3')
        spanTs = 3;
    elseif strcmp(pulse_type, 'sinc5')
        spanTs = 5;
    else
        spanTs = [];
    end

    [t, y_rec] = reconstruct_signal(n, y, Ts, pulse_type, spanTs);

    figure;
    plot(t, y_rec); hold on;
    stem(n*Ts, y, 'filled');
    grid on;

    xlabel('t (seconds)');
    ylabel('Amplitude');
    title([seq_label, ' using ', pulse_type]);
    legend('y(t)', 'y[n]');
end
```

Comparing Sinc3 and Sinc5

This function is written for comparing Sinc3 and Sinc5 on the same figure.

```
function compare_sinc_truncation(n, y, Ts, seq_label)
    [t3, y3] = reconstruct_signal(n, y, Ts, 'sinc3', 3);
    [t5, y5] = reconstruct_signal(n, y, Ts, 'sinc5', 5);

    figure;
    plot(t3, y3); hold on;
    plot(t5, y5);
    grid on;
    xlabel('t (seconds)');
    ylabel('Amplitude');
    title([seq_label ' Sinc truncation comparison']);
    legend('+/- 3Ts', '+/- 5Ts');
end
```

Reconstruct Signal

This is the main function which is responsible for the signal reconstruction. First, it adjusts the *pulse_span_Ts* for the pulse type given as parameter.

```
function [t, y_rec] = reconstruct_signal(n, y, Ts, pulse_type, spanTs)
    % pulse support in Ts units
    switch pulse_type
        case 'square'
            pulse_span_Ts = 1;
        case 'triangular'
            pulse_span_Ts = 2;
        case 'parabolic'
            pulse_span_Ts = 4;
        case 'sinc3'
            pulse_span_Ts = 2 * 3;
        case 'sinc5'
            pulse_span_Ts = 2 * 5;
    end
```

In the following, L is the oversampling factor. We choose L = 50 to generate 50 points per sampling period Ts. This is required to make the signal look more continuous rather than discrete. Moreover, since L is the number of points per period *Ts*, dividing *Ts* by *L* gives the time increment *dt* between these points.

```
    % time axis
    L  = 50;
    dt = Ts / L;
    t_min = (min(n) - pulse_span_Ts/2) * Ts;
    t_max = (max(n) + pulse_span_Ts/2) * Ts;
    t = t_min : dt : t_max;
```

Finally, in the following, one of the pulse functions is called with calculated *tk* and *Ts*. Here, *tk* represents the shifted time vector. This shift is necessary to center the pulse at the specific sample instance *n[k]*.

```
    % reconstruction
    y_rec = zeros(size(t));
    for k = 1:length(n)
        tk = t - n(k)*Ts;
        switch pulse_type
            case 'square'
                p = pulse_square(tk, Ts);
            case 'triangular'
                p = pulse_triangular(tk, Ts);
            case 'parabolic'
                p = pulse_parabolic(tk, Ts);
            case {'sinc3','sinc5'}
                p = pulse_sinc(tk, Ts, spanTs);
```

```
        end
        y_rec = y_rec + y(k) * p;
    end
end
```

Pulse Functions

The following are pulse functions that generates the pulse signal.

Square Pulse:

$$p(t) = \begin{cases} 1 & \text{for } -\frac{T_s}{2} \leq t \leq \frac{T_s}{2} \\ 0 & \text{otherwise} \end{cases}$$

```
function p = pulse_square(t, Ts)
    p = double( (t >= -Ts/2) & (t <= Ts/2) );
end
```

Triangular Pulse:

$$p(t) = \begin{cases} 1 & \text{for } -\frac{T_s}{2} \leq t \leq \frac{T_s}{2} \\ 0 & \text{otherwise} \end{cases}$$

```
function p = pulse_triangular(t, Ts)
    p = max(0, 1 - abs(t)/Ts);
end
```

Parabolic Pulse: This function generates a pulse consisting of three parabolic segments to approximate the ideal Sinc pulse. It creates three segments: middle, left, and right. For the middle segment, the parabola mentioned in the comment section is used. For the left and right segments, to match the negative side lobe of the Sinc function, a quadratic equation is derived with roots at Ts and 2Ts. The factor 0.85 scales the depth of this negative side lobe to approximate the Sinc function's minimum.

```
function p = pulse_parabolic(t, Ts)
    p = zeros(size(t));


    % middle segment, 1 - (t/Ts)^2
    mid = (abs(t) <= Ts);
    p(mid) = 1 - (t(mid)/Ts).^2;

    % left segment
    left = (t >= -2*Ts) & (t < -Ts);
    tl = t(left);
```

```
    % A * (x - r1) * (x - r2)
    p(left) = 0.85 * (tl/Ts + 1) .* (tl/Ts + 2);

    % right segment
    right = (t > Ts) & (t <= 2*Ts);

    if sum(left) == sum(right)
        p(right) = flip(p(left));
    else
        tr = t(right);
        p(right) = 0.85 * (tr/Ts - 1) .* (tr/Ts - 2);
    end
end
```

Sinc pulse:

$$p(t) = \text{sinc}\left(\frac{t}{T_s}\right) = \frac{\sin(\pi t/T_s)}{\pi t/T_s}$$

```
function p = pulse_sinc(t, Ts, spanTs)
    p = zeros(size(t));
    idx = abs(t) <= spanTs*Ts;
    x = pi * t(idx) / Ts;
    p(idx) = sin(x) ./ x;
    p(t == 0) = 1;
end
```

The Second Version (works with user input)

The following is the second version of the explained code which runs based on user input.

```
clear
clc
close all

modeChoice = input('Select mode (1 demonstration of a, b, c; 2 trying your own
sequence): ');

Ts = input('Enter the sampling period Ts: ');

pulseNumber = menu('Select pulse type', ...
    'Square', 'Triangular', 'Parabolic', 'Truncated ideal');

switch pulseNumber
    case 1
        pulse = 'square';
    case 2
```

```
        pulse = 'triangular';
    case 3
        pulse = 'parabolic';
    case 4
        pulse = 'sinc3';
end

if modeChoice == 1
    % Sequence a
    n_a = -3:6;
    y_a = [2, -2, 0, 2, 0, 1, 0, -5, 7, 0]; % id number values

    % Sequence b
    n_b = -12:12;
    y_b = 10 * exp(-0.25 * abs(n_b));

    % Parts a and b
    reconstruct_and_plot(n_a, y_a, Ts, pulse, 'Sequence (a)');
    reconstruct_and_plot(n_b, y_b, Ts, pulse, 'Sequence (b)');

    % Part c
    if strcmp(pulse, 'sinc3')
        compare_sinc_truncation(n_a, y_a, Ts, 'Sequence (a)');
        compare_sinc_truncation(n_b, y_b, Ts, 'Sequence (b)');
    end
elseif modeChoice == 2
    n = input('Enter index vector n: ');
    y = input('Enter sequence y[n] (same length as n): ');

    n = n(:).';
    y = y(:).';

    if numel(n) ~= numel(y)
        error('Error: n and y must have the same length.');
    end

    reconstruct_and_plot(n, y, Ts, pulse, 'User-defined sequence');

    if strcmp(pulse, 'sinc3')
        compare_sinc_truncation(n, y, Ts, 'User-defined sequence');
    end
end

function reconstruct_and_plot(n, y, Ts, pulse_type, seq_label)
    if strcmp(pulse_type, 'sinc3')
        spanTs = 3;
    elseif strcmp(pulse_type, 'sinc5')
        spanTs = 5;
    else
        spanTs = [];
```

```
    end

    [t, y_rec] = reconstruct_signal(n, y, Ts, pulse_type, spanTs);

    figure;
    plot(t, y_rec); hold on;
    stem(n*Ts, y, 'filled');
    grid on;

    xlabel('t (seconds)');
    ylabel('Amplitude');
    title([seq_label, ' using ', pulse_type]);
    legend('y(t)', 'y[n]');
end

function compare_sinc_truncation(n, y, Ts, seq_label)
    [t3, y3] = reconstruct_signal(n, y, Ts, 'sinc3', 3);
    [t5, y5] = reconstruct_signal(n, y, Ts, 'sinc5', 5);

    figure;
    plot(t3, y3); hold on;
    plot(t5, y5);
    grid on;
    xlabel('t (seconds)');
    ylabel('Amplitude');
    title([seq_label ' Sinc truncation comparison']);
    legend('+/- 3Ts', '+/- 5Ts');
end

function [t, y_rec] = reconstruct_signal(n, y, Ts, pulse_type, spanTs)
    % pulse support in Ts units
    switch pulse_type
        case 'square'
            pulse_span_Ts = 1;
        case 'triangular'
            pulse_span_Ts = 2;
        case 'parabolic'
            pulse_span_Ts = 4;
        case 'sinc3'
            pulse_span_Ts = 2 * 3;
        case 'sinc5'
            pulse_span_Ts = 2 * 5;
    end

    % time axis
    L  = 50;
    dt = Ts / L;
    t_min = (min(n) - pulse_span_Ts/2) * Ts;
    t_max = (max(n) + pulse_span_Ts/2) * Ts;
    t = t_min : dt : t_max;
```

```
    % reconstruction
    y_rec = zeros(size(t));
    for k = 1:length(n)
        tk = t - n(k)*Ts;
        switch pulse_type
            case 'square'
                p = pulse_square(tk, Ts);
            case 'triangular'
                p = pulse_triangular(tk, Ts);
            case 'parabolic'
                p = pulse_parabolic(tk, Ts);
            case {'sinc3','sinc5'}
                p = pulse_sinc(tk, Ts, spanTs);
        end
        y_rec = y_rec + y(k) * p;
    end
end

function p = pulse_square(t, Ts)
    p = double( (t >= -Ts/2) & (t <= Ts/2) );
end

function p = pulse_triangular(t, Ts)
    p = max(0, 1 - abs(t)/Ts);
end

function p = pulse_parabolic(t, Ts)
    p = zeros(size(t));


    % middle segment, 1 - (t/Ts)^2
    mid = (abs(t) <= Ts);
    p(mid) = 1 - (t(mid)/Ts).^2;

    % left segment
    left = (t >= -2*Ts) & (t < -Ts);
    tl = t(left);

    % A * (x - r1) * (x - r2)
    p(left) = 0.85 * (tl/Ts + 1) .* (tl/Ts + 2);

    % right segment
    right = (t > Ts) & (t <= 2*Ts);

    if sum(left) == sum(right)
        p(right) = flip(p(left));
    else
        tr = t(right);
        p(right) = 0.85 * (tr/Ts - 1) .* (tr/Ts - 2);
```

```
    end
end

function p = pulse_sinc(t, Ts, spanTs)
    p = zeros(size(t));
    idx = abs(t) <= spanTs*Ts;
    x = pi * t(idx) / Ts;
    p(idx) = sin(x) ./ x;
    p(t == 0) = 1;
end
```